1. 개발 환경 구성

이 절에서는 자바 프로그램을 작성하고 실행하기 위해 필요한 Zulu Open JDK 21 설치 과정과 IDE 환경 설정 방법을 다루며, 실무와 교육에서 널리 활용되는 개발 도구인 IntelliJ IDEA를 설치하는 과정을 익힙니다.

(1) 자바 개발 환경 구축의 개요

비용이나 라이선스를 감안하여 무료 배포판인 Azul Zulu Builds of OpenJDK Zulu 21 LTS(Long Term Support) 기반으로 안정성·보안 패치가 장기적으로 지원되는 버전을 설치하고, 코드 편집과 프로젝트 관리를 위한 통합 개발 환경(IDE: Integrated Development Environment)을 제공하는 IntelliJ Community를 설치합니다.

① 개발 환경 구성 흐름

온전한 프로젝트 구성과 JDK 인식을 위해 반드시 구성 흐름의 순서를 지켜야 합니다. 우선, JDK를 설치한 후 어느 디렉터리(폴더)에서도 Java가 인식될 수 있게 경로(Path)를 설정을 하여야 합니다. 그래야 IntelliJ Community에서 새로운 프로젝트를 추가했을 때 앞서 설정한 JDK를 불러 올 수 있습니다. 그럼 순서대로 개발 환경 구성의 흐름을 알아보겠습니다.

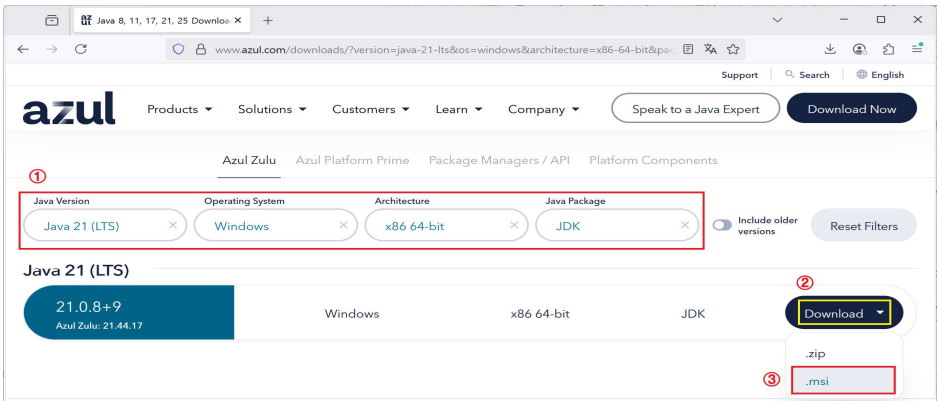
> 1. **Zulu OpenJDK 21 설치** → 2. **Java Home & Path 설정** → 3. **IntelliJ 다운로드** → 4. **IntelliJ 설치** → 5. **IntelliJ에서 JDK 연결 및 새 Java 프로젝트 생성**

(2) Windows 시스템에 JDK 다운로드 및 설치

지금부터는 윈도우즈(Windows) 운영체제가 설치되어 있는 환경에서 Java를 다운로드 받아 설치하는 작업을 진행할 것입니다. 그리고 이후에 자바 개발 환경 설정을 이어가겠습니다.

① Zulu OpenJDK 21 다운로드

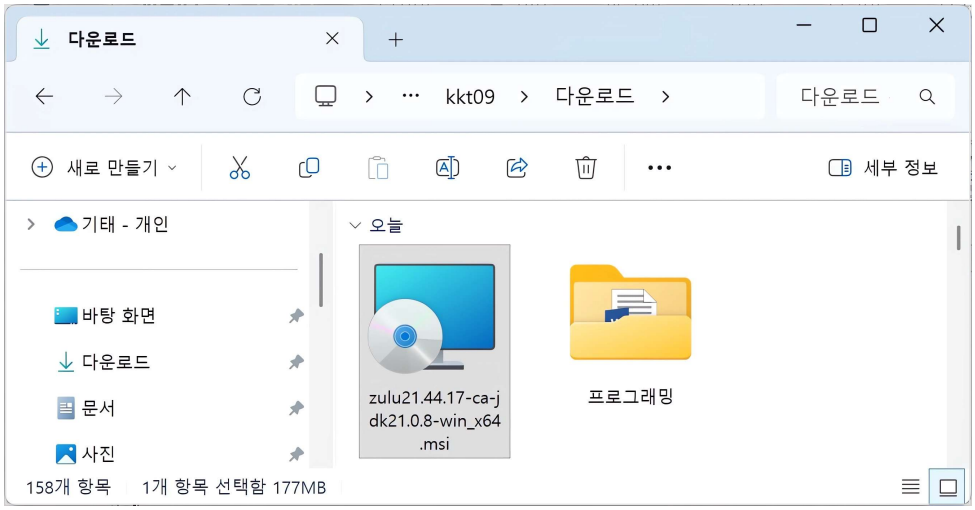
Azul 다운로드 페이지(https://www.azul.com/downloads/)에 방문하여 아래와 같이 Zulu (OpenJDK)에서 Java 21 (Windows) 항목을 선택하고 MSI (x64)를 다운로드합니다.(화면은 다운로드 페이지에서 아래로 스크롤 해야함)



[그림 1] zulu JDK 21 다운로드

② Zulu OpenJDK 21 설치 관리자 실행

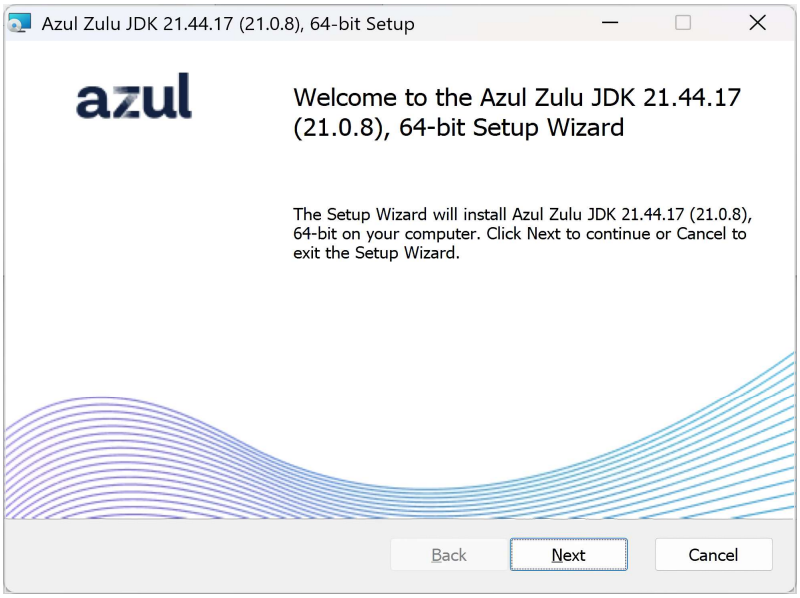
다운로드한 zulu21.xx.xx-...msi 파일을 더블클릭하여 설치 관리자를 실행합니다.



[그림 2] 자바 설치 파일

③ Zulu OpenJDK 21 설치 관리자 시작

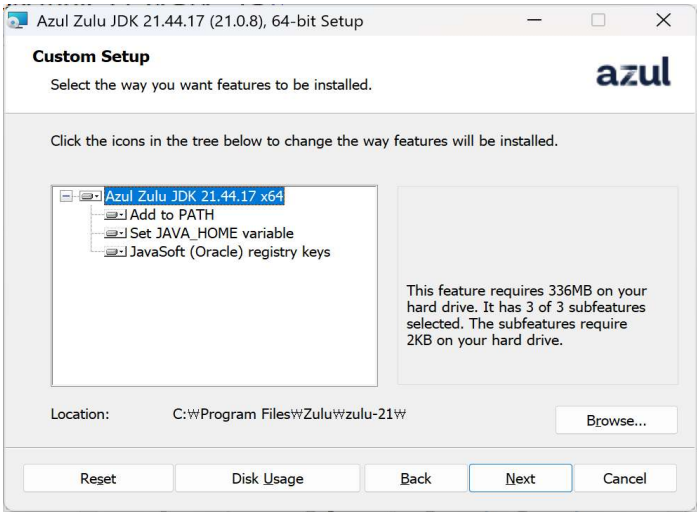
Azul Zulu JDK 21의 Setup Wizard 화면이 나타나면 [Next] 버튼을 눌러 설치를 진행합니다.



[그림 3] zulu OpenJ 21 설치 관리자

④ Zulu OpenJDK 21 사용자 설정

“Add to PATH”, “Set JAVA_HOME variable”, “JavaSoft (Oracle registry keys” 항목 앞의 펼침 아이콘을 클릭하여 모두 “Will be installed on logical hard drive”를 선택하여 로컬에 설치되고 설정될 수 있게 지정, Location(설치 위치)를 원하는 위치로 변경(여기에서는 기본 위치인 “C:₩Program Files₩Zulu₩zulu-21₩” 위치)한 후 [Next] 버튼을 누릅니다. 중간 설치 단계의 화면에서도 [Install] 버튼을 누릅니다.



[그림 4] zulu OpenJDK 21 사용자 설정

⑤ Zulu OpenJDK 21 설치 완료

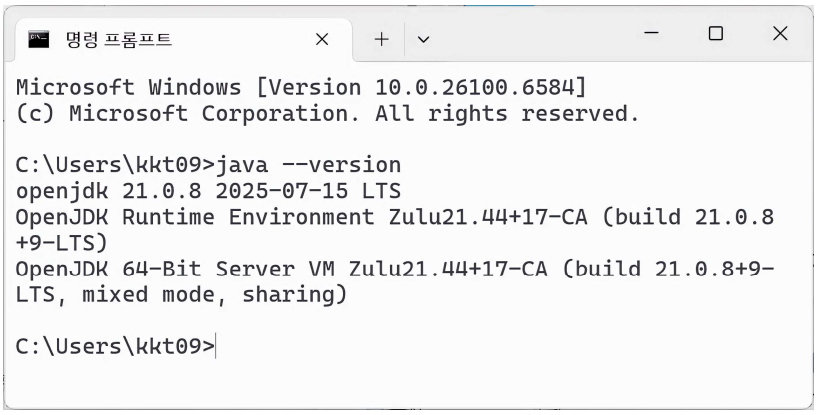
Zulu OpenJDK 21의 설치 완료 화면이 나타나면 [Finish] 버튼을 눌러 설치를 완료합니다.



[그림 5] zulu OpenJDK 21 설치 완료

⑥ 자바 설치 확인

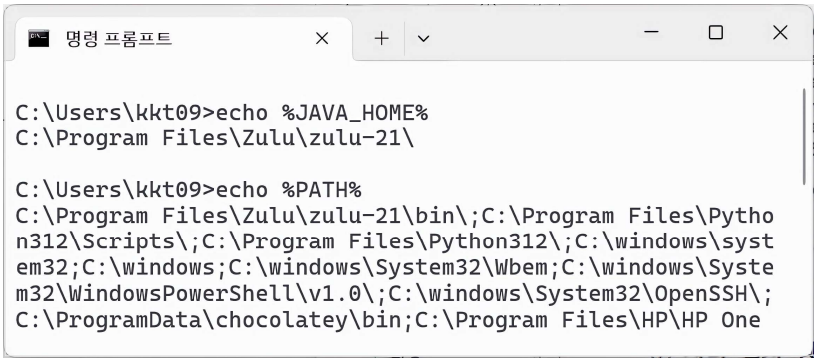
명령 프롬프트를 실행한 후 “java –version” 명령을 입력하여 자바가 제대로 설치되었는지 확인합니다. 아래 그림과 같이 자바의 설치된 내용이 출력되면, 설치가 성공적으로 완료된 것입니다.



[그림 6] 자바 설치 확인

⑦ 자바 설치 경로 확인

명령 프롬프트에서 “echo %JAVA_HOME%” 명령으로 JAVA_HOME이 설정이 되었는지, “echo %PATH%” 명령으로 경로 설정이 되었는지 아래 그림과 같이 출력내용을 확인합니다.

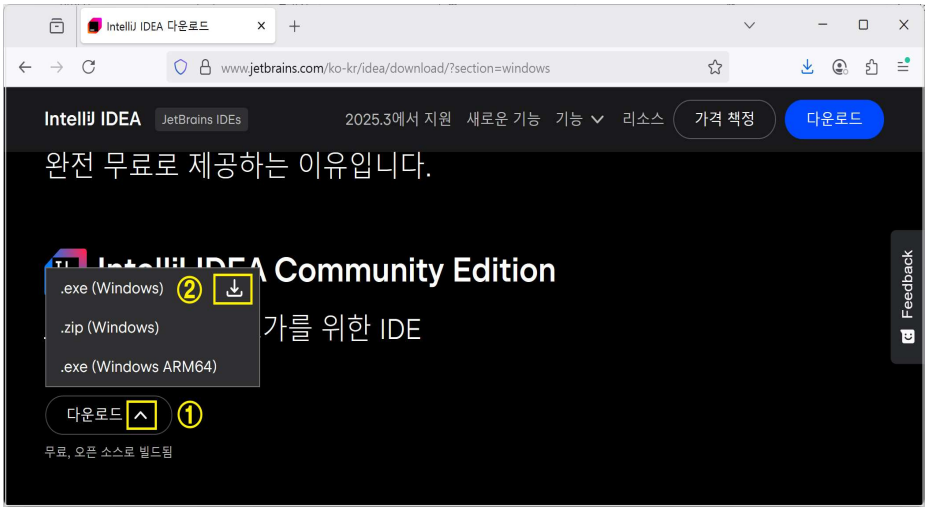


[그림 7] 자바 설치 경로 확인

(2) IDE 설정 (IntelliJ IDEA)

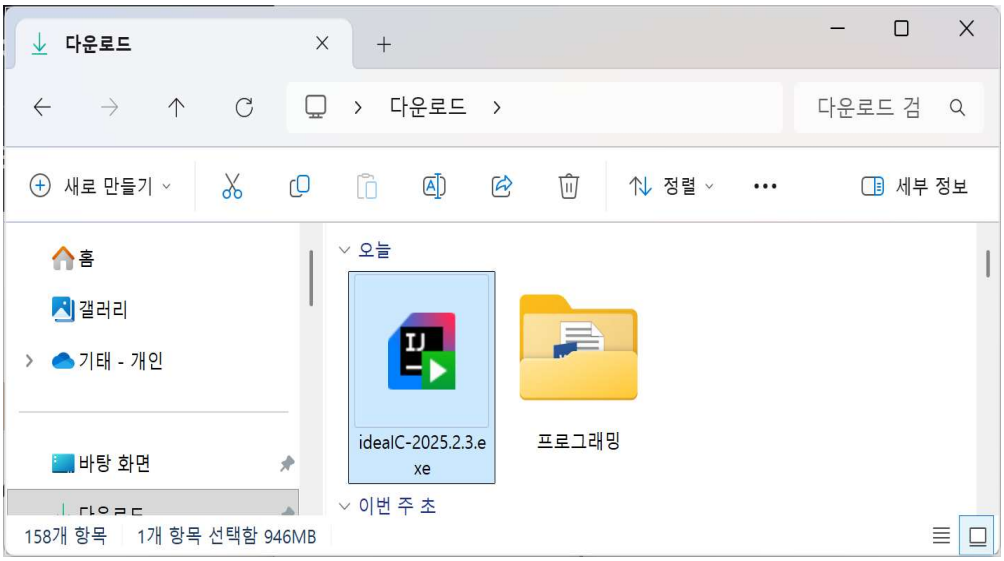
Spring Boot 개발에 가장 널리 사용되는 IDE는 JetBrains의 IntelliJ IDEA와 Eclipse 기반의 Spring Tool Suite(STS)입니다. 두 IDE 모두 Spring Boot 개발을 훌륭하게 지원하지만, IntelliJ IDEA가 생산성과 개발 편의성 면에서 현업에서 더 선호됩니다.

① JetBrains 공식 다운로드 페이지(https://www.jetbrains.com/ko-kr/idea/download/)에서 화면을 아래로 스크롤하여 IntelliJ IDEA Community 버전 다운로드 화면으로 이동한 후 다운로드 버튼의 오른쪽 펼침 아이콘을 클릭하여 ".exe (Windows)" 항목을 선택하여 다운로드 받습니다.



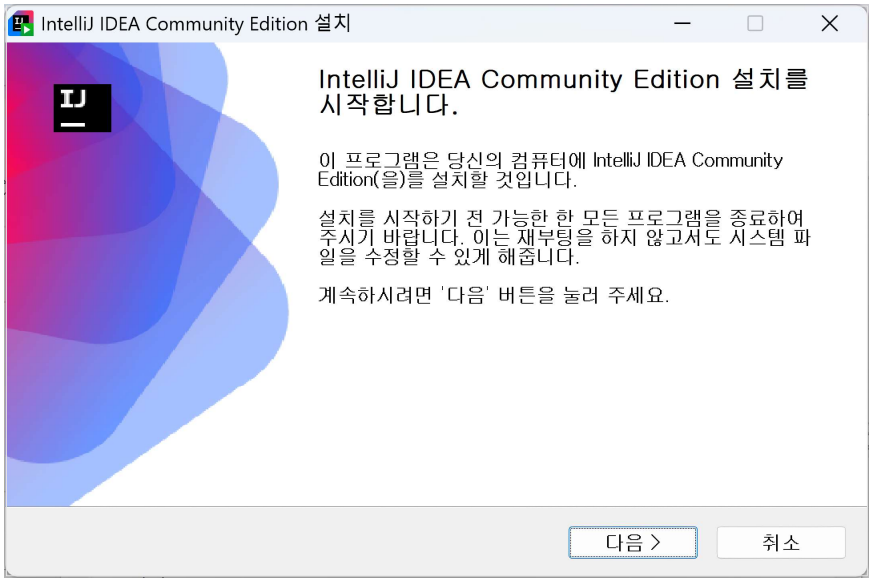
[그림 8] IntelliJ Community 다운로드 페이지

② 다운로드 폴더로 이동하여 "idealC-2025.x...exe"를 더블클릭하여 설치를 시작합니다.



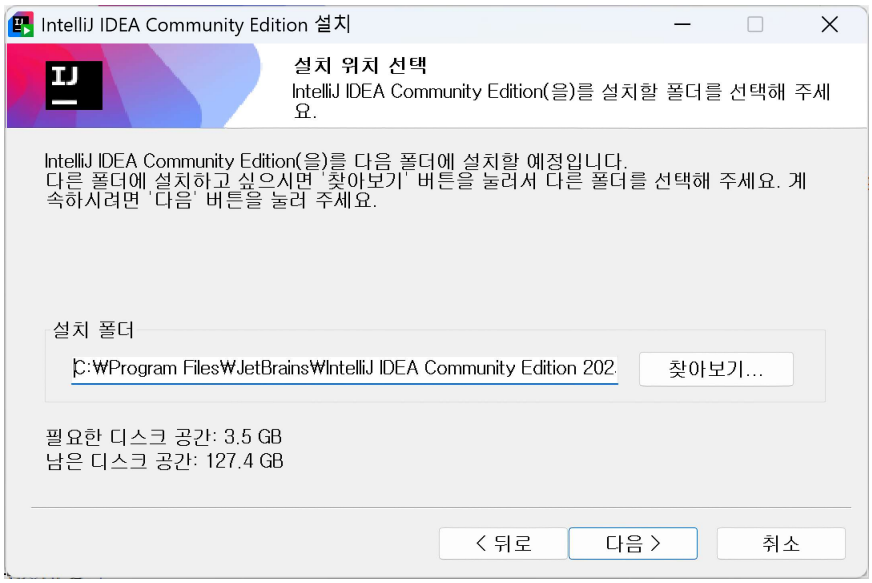
[그림 9] IntelliJ Community 설치 프로그램

③ IntelliJ IDEA Communication Edition 설치 화면이 시작되면, [다음] 버튼을 눌러 설치를 시작합니다.



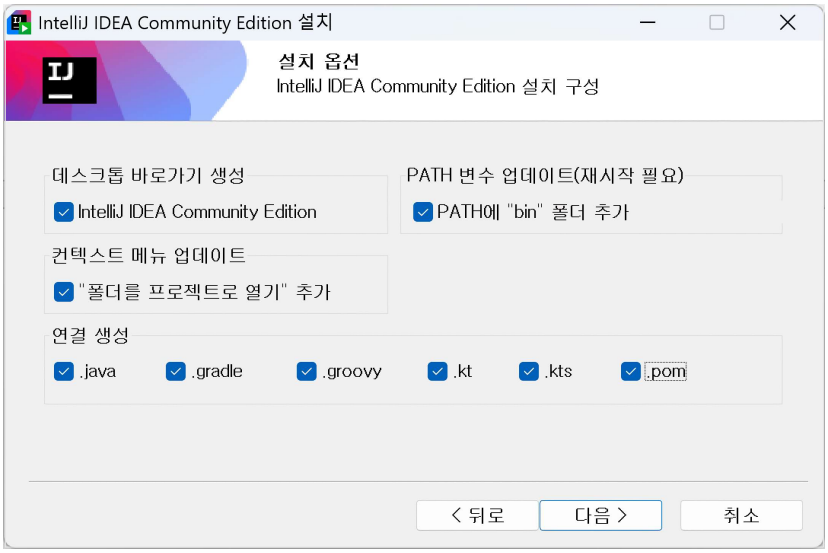
[그림 10] IntelliJ Community 설치 웰컴

④ IntelliJ IDEA Communication Edition 설치의 설치 위치 선택 화면이 나타나면, [찾아보기...] 버튼을 눌러 설치할 디렉터리(폴더)를 지정(여기에서는 기본값인 "C:₩Program Files₩JetBrains₩IntelliJ IDEA Community Edition 2025.2.3."으로 함)하고, [다음 >] 버튼을 눌러 설치를 진행합니다.



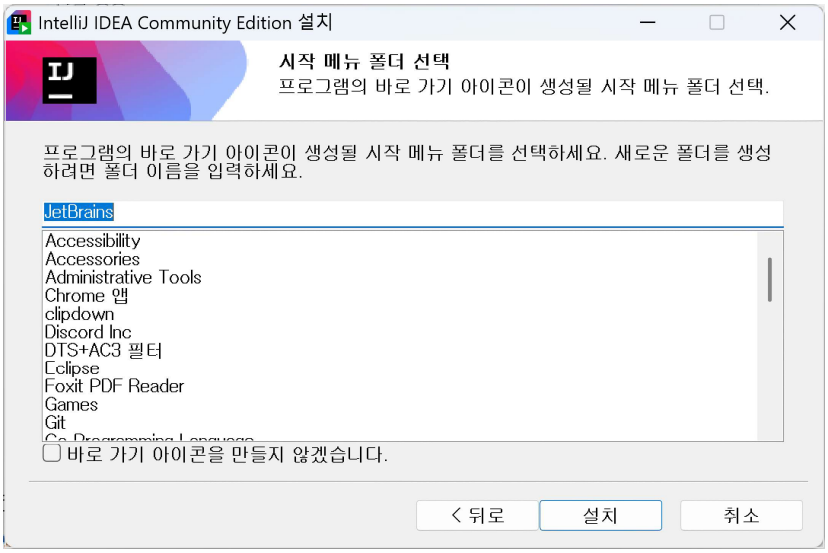
[그림 11] IntelliJ Community 설치 위치 선택

⑤ IntelliJ IDEA Communication Edition 설치의 설치 옵션 화면이 나타나면, 아래 그림과 같이 모든 옵션의 체크박스를 체크한 후 [다음 〉] 버튼을 눌러 설치를 진행합니다.



[그림 12] IntelliJ Community 설치 옵션

⑥ IntelliJ IDEA Communication Edition 설치의 시작 메뉴 폴더 선택 화면이 나타나면, 시작 메뉴에 나타날 메뉴명을 확인한 후 [설치] 버튼을 눌러 설치를 진행합니다.



[그림 13] IntelliJ Community 시작 메뉴 폴더 선택

⑦ IntelliJ IDEA Communication Edition 설치의 설치 완료 화면이 나타나면, “IntelliJ IDEA Communication Edition 실행하기”를 체크하고, [마침] 버튼을 눌러 설치를 완료하고, IntelliJ를 실행합니다.



[그림 14] IntelliJ Community 설치 완료

(3) IntelliJ IDEA 설정

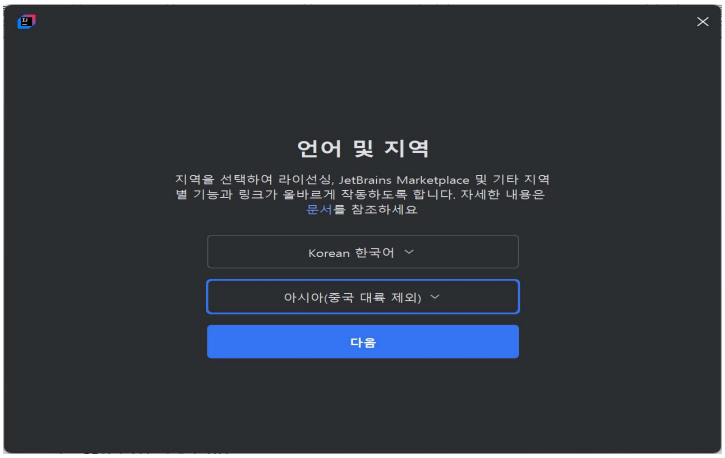
전 세계 스프링 개발자들이 가장 선호하는 IDE인 IntelliJ IDEA는 단순한 텍스트 에디터를 넘어 코드 분석, 자동 완성, 강력한 리팩토링 기능을 제공합니다. 스프링 부트 애플리케이션 개발에 최적화된 IDE 환경 설정을 통해, 코드 작성 외의 불필요한 작업 시간을 줄이는 최적의 세팅법을 학습합니다.

- ✓ IntelliJ IDEA Ultimate: Spring 관련 플러그인이 기본 내장되어 있어 가장 강력한 Spring 개발 환경 제공 (유료/학생 무료)
- ✓ IntelliJ IDEA Community: 무료 버전이지만 Spring Boot 플러그인 별도 설치 필요 ("Spring Boot" 플러그인 검색 후 설치)

IntelliJ IDEA를 Spring Boot 개발에 최적화하기 위한 주요 설정

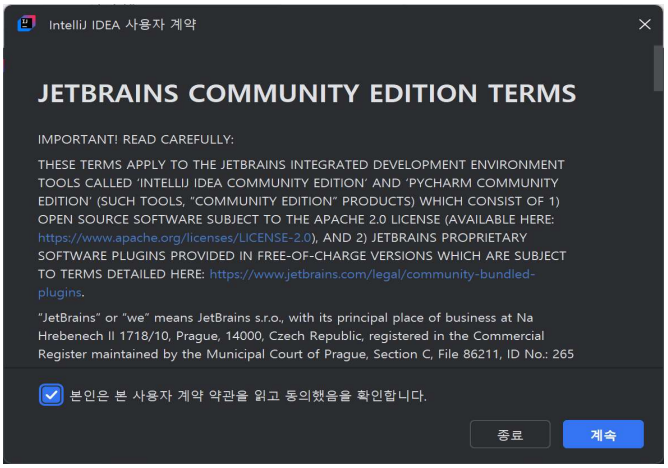
- ✓ File > Settings > Build, Execution, Deployment > Compiler > Build project automatically 체크
- ✓ File > Settings > Advanced Settings > Allow auto-make to start even if developed application is currently running 체크 (DevTools 자동 재시작 활성화)
- ✓ File > Settings > Editor > General > Auto Import > Add unambiguous imports on the fly 체크 (자동 import)
- ✓ File > Settings > Editor > Code Style > Java > Tabs and Indents: Use tab character 해제, Tab size 4, Indent 4 설정

① IntelliJ를 처음 실행할 경우 언어 및 지역 화면이 나타납니다. 아래 그림과 같이 언어는 “Korean 한국어”로 지역 선택은 “아시아(중국 대륙제외)”를 선택한 후 [다음] 버튼을 누릅니다. (처음 한 번만 표시됩니다.)



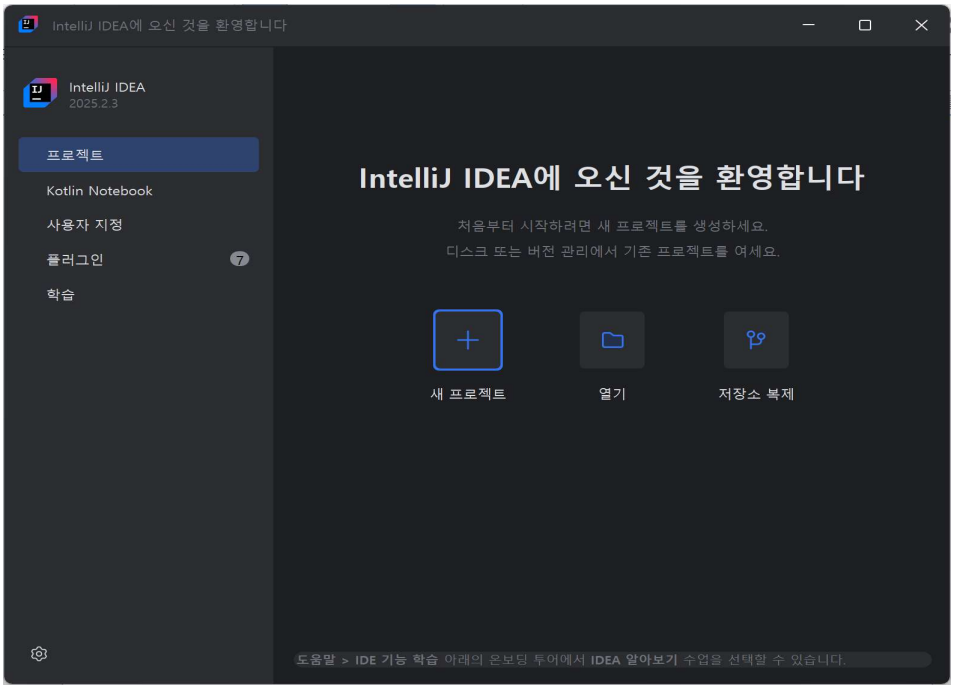
[그림 15] IntelliJ Community 언어 및 지역 설정

② IntelliJ IDEA 사용자 계약 화면이 나타나면, “본인은 본 사용자 계약 약관을 읽고 동의했음을 확인합니다.”를 체크한 후 [계속] 버튼을 누릅니다. (설치 시 처음 한 번만 표시됩니다.)



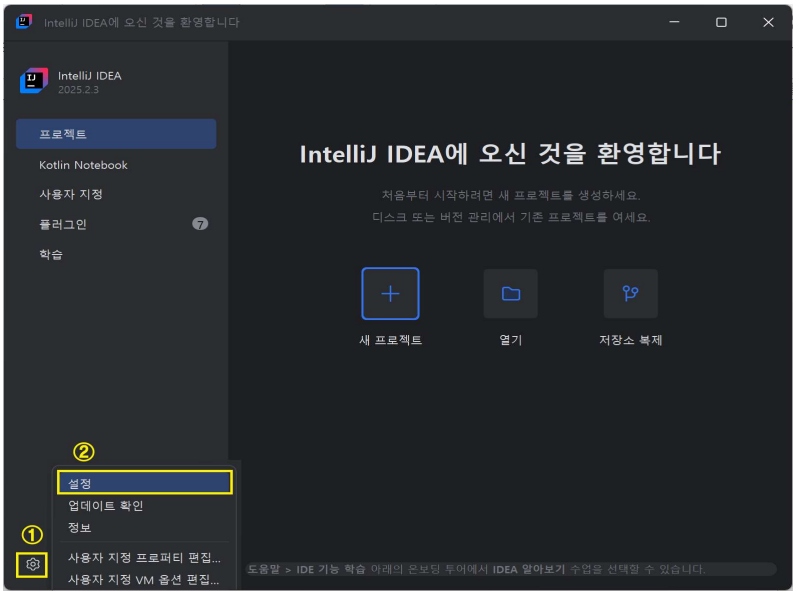
[그림 16] IntelliJ Community 사용자 계약

③ 제대로 실행되고 나면, IntelliJ IDEA의 웰컴 화면이 아래와 같이 나타납니다.



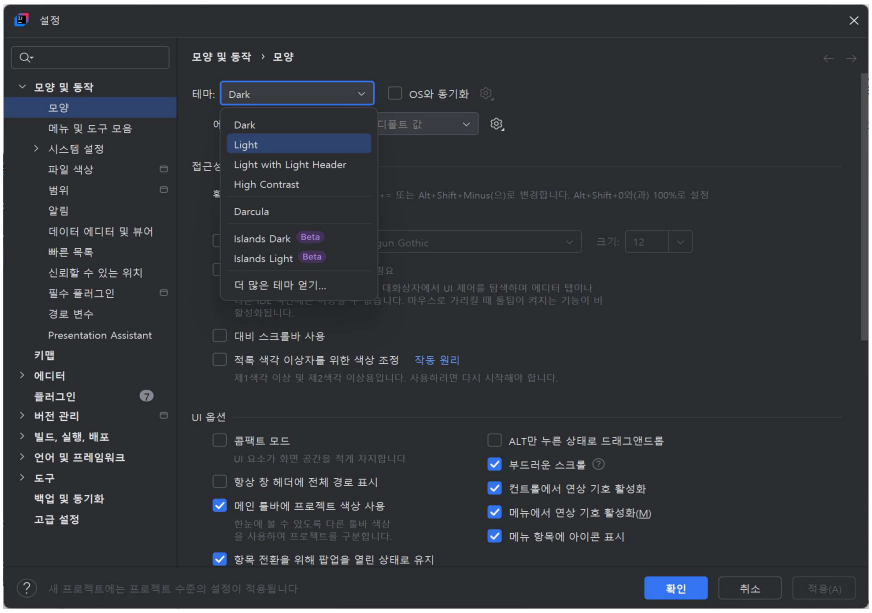
[그림 17] IntelliJ Community 웰컴 화면

④ 설정을 위해 화면 왼쪽 하단의 톱니바퀴 모양의 아이콘을 클릭하면 나오는 메뉴에서 [설정] 메뉴를 선택합니다.



[그림 18] IntelliJ Community 설정 메뉴

⑤ 설정 대화상자에서 [모양 및 동작]-[모양] 메뉴에서 테마를 “Light”로 변경하면, 테마가 변경됩니다.



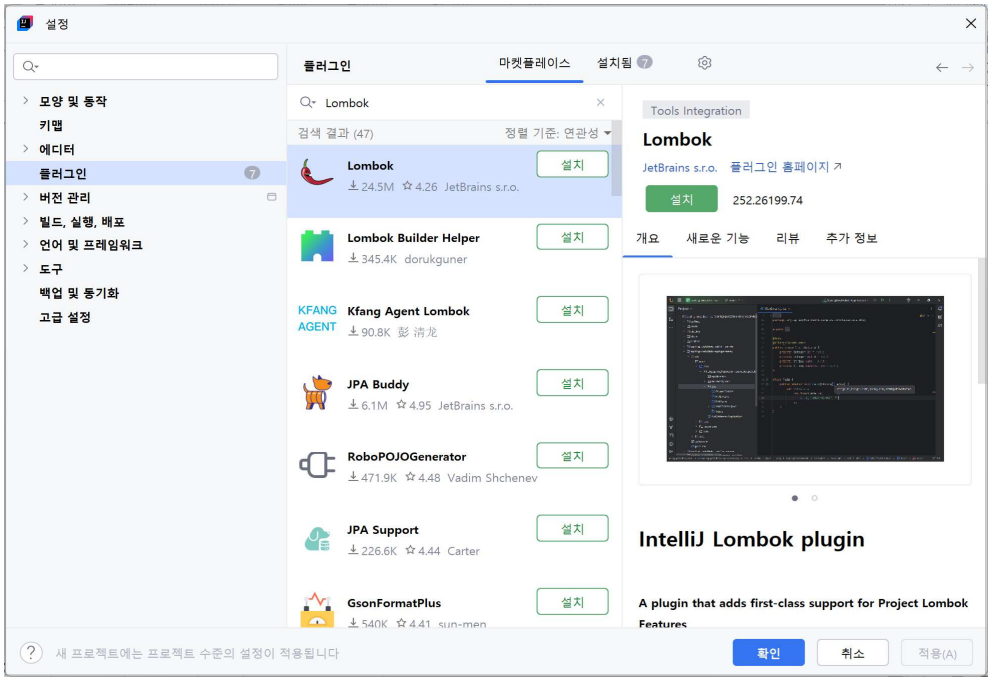
[그림 19] IntelliJ Community 테마 설정

⑥ 설정 대화상자에서 왼쪽의 [플러그인] 메뉴를 선택하고, [마켓플레이스] 탭 아이콘을 클릭하면, 검색 창이 열리는데 이 곳에 “Java”를 입력하고, [설치] 버튼을 눌러 Java 플러그인을 설치합니다.(만약, [설치] 버튼이 없다면, 이미 설치되어 있으므로 설치할 필요없음)



[그림 20] IntelliJ Community - Java 플러그인 설치

⑦ 설정 대화상자에서 [플러그인] 메뉴가 선택되어 있는 상태에서 “Lombok”을 검색하여 Lombok의 [설치] 버튼을 눌러 설치합니다. 그리고, 더 이상 설치할 플러그인이 없으면, [확인] 버튼을 눌러 설정을 완료합니다.



[그림 21] IntelliJ Community - Lombok 플러그인 설치

⑧ 유용한 IntelliJ IDEA 플러그인

기본 기능만으로도 훌륭한 IntelliJ이지만, 적절한 플러그인의 조합은 여러분의 개발 경험을 한 단계 더 높여줍니다. 반복적인 코드를 줄여주는 Lombok부터 코드 품질을 실시간으로 점검하는 도구까지, 현업 개발자들이 필수적으로 사용하는 플러그인 생태계를 소개합니다.

플러그인명	기능
Lombok	@Getter, @Setter, @Builder 등 Lombok 어노테이션 코드 자동 생성 지원
Spring Boot Assistant	application.yml 자동 완성 및 Spring Boot 설정 지원
GitToolBox	Git blame, 자동 fetch 등 Git 생산성 향상
SonarLint	코드 품질 분석 및 버그 가능성 있는 코드 실시간 경고
Key Promoter X	단축키 학습 지원으로 생산성 향상
Rainbow Brackets	중첩된 괄호를 색상별로 표시하여 가독성 향상

(4) Gradle / Maven 소개

빌드 도구(Build Tool)는 소스 코드 컴파일, 의존성 관리, 패키징, 테스트 실행 등 소프트웨어 개발 프로세스를 자동화합니다. Spring Boot 프로젝트에서는 Gradle과 Maven 중 하나를 선택하여 사용합니다.

비교 항목	Gradle	Maven
설정 파일	build.gradle (Groovy/Kotlin DSL)	pom.xml (XML)
빌드 성능	증분 빌드, 빌드 캐시 지원으로 Maven 대비 빠름	전체 빌드 방식으로 상대적으로 느림
유연성	스크립트 기반으로 높은 커스터마이징 가능	선언적 방식으로 관례를 따름
학습 곡선	Groovy/Kotlin 지식 필요로 다소 높음	XML 기반으로 직관적, 낮은 편
현업 트렌드	Android, Spring Boot 신규 프로젝트에서 선호	레거시 프로젝트 및 기업 환경에서 여전히 광범위
Spring Initializr 기본	Gradle - Groovy 기본 제공	Maven도 선택 가능

실무 권장	신규 Spring Boot 프로젝트는 Gradle을 사용하는 것이 일반적인 추세입니다. 특히 멀티 모듈 프로젝트에서 Gradle의 강점이 두드러집니다. 그러나 기존 Maven 프로젝트를 유지보수해야 하는 경우가 많으므로 두 도구 모두 이해하는 것을 권장합니다.

2. Spring Initializr를 통한 프로젝트 생성

과거에는 프로젝트 하나를 시작하기 위해 수십 개의 라이브러리 버전을 일일이 맞추고 복잡한 빌드 스크립트를 작성해야 했습니다. 1.5 섹션에서는 스프링 부트의 공식 프로젝트 생성 도구인 Spring Initializr를 활용하여, 최신 Java 21 환경에 최적화된 프로젝트의 뼈대를 순식간에 구성하는 방법을 배웁니다.

(1) Spring Initializr 사용 방법

Spring Initializr(https://start.spring.io)는 Spring Boot 프로젝트의 초기 설정을 자동으로 생성해 주는 공식 웹 서비스입니다. 복잡한 빌드 설정 없이 몇 번의 클릭만으로 바로 개발을 시작할 수 있는 프로젝트 구조를 생성해 줍니다. IntelliJ IDEA Ultimate에서는 IDE 내에서 직접 Spring Initializr를 사용하여 프로젝트를 생성할 수도 있습니다.

Spring Initializr에서 설정하는 주요 항목들은 다음과 같습니다.

설정 항목	설명	권장 선택값 (2024년 기준)
Project	빌드 도구 선택	Gradle - Groovy (또는 Kotlin DSL)
Language	프로그래밍 언어	Java
Spring Boot	Spring Boot 버전	3.5.x (현재 최신 GA 버전)
Group	프로젝트 그룹 ID (역방향 도메인)	com.example (실제 도메인으로 변경)
Artifact	프로젝트의 고유 식별자 (Artifact ID)	myapp (소문자, 하이픈 사용)
Name	프로젝트 이름 (Artifact와 동일하게 설정)	myapp
Description	프로젝트 설명	자유 입력
Package name	기본 패키지명 (Group + Artifact 자동 생성)	com.example.myapp
Packaging	패키징 방식	Jar (Spring Boot 기본, 내장 서버 포함)
Java	Java 버전	21 (LTS)

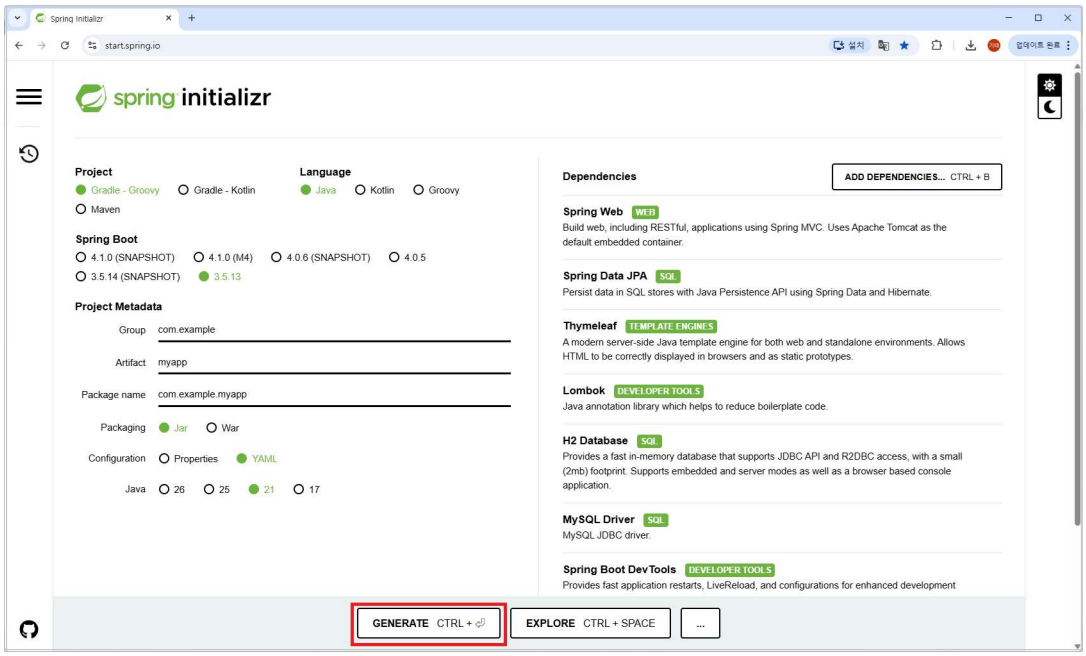
(2) 의존성(Dependencies) 설정

Spring Initializr에서 프로젝트에 필요한 의존성을 검색하고 추가할 수 있습니다. 이 책의 학습 과정에서 사용할 주요 의존성들과 그 역할을 정리합니다.

의존성 이름	Starter 명칭	용도
Spring Web	spring-boot-starter-web	Spring MVC 기반 웹 개발, 내장 Tomcat 포함
Spring Data JPA	spring-boot-starter-data-jpa	JPA/Hibernate 기반 데이터베이스 연동
Thymeleaf	spring-boot-starter-thymeleaf	서버 사이드 HTML 템플릿 엔진
Lombok	lombok	반복적인 Java 코드(Getter, Setter 등) 어노테이션으로 생성

의존성 이름	Starter 명칭	용도
H2 Database	h2	인메모리 데이터베이스 (개발/테스트용)
MySQL Driver	mysql-connector-j	MySQL 데이터베이스 드라이버
Spring Boot DevTools	spring-boot-devtools	개발 시 자동 재시작 등 개발 생산성 향상
Validation	spring-boot-starter-validation	Bean Validation API (입력값 검증)
Spring Reactive Web	spring-boot-starter-webflux	Reactive(반응형) 웹 개발 (WebFlux)

왼쪽에는 Spring Initializr의 프로젝트 정보가 입력되며, 오른쪽에는 [ADD DEPENDENCIES...] 버튼을 눌러 의존성 설정표를 참고하여 의존성 라이브러리를 추가합니다. 모두 완료되면,
아래 그림과 같이 위 주요 항목과 의존성의 내용으로 확인한 후 [GENERATE] 버튼을 누릅니다.



[그림 22] 스프링 이니셜라이저

생성된 build.gradle 파일 예시는 다음과 같습니다.

Groovy - build.gradle 예시

```
// build.gradle (Groovy DSL)
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.13'
    id 'io.spring.dependency-management' version '1.1.6'
```

```
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
configurations {
    compileOnly {
        extendsFrom annotationProcessor
    }
}
repositories {
    mavenCentral()
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    compileOnly 'org.projectlombok:lombok'
    runtimeOnly 'com.h2database:h2'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') {
    useJUnitPlatform()
}
```